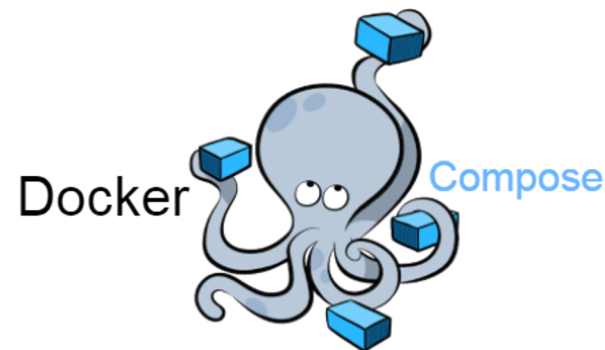
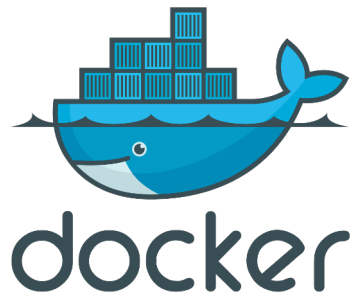


Ingeniería del Software 2

Contenedores con Docker y Docker Compose





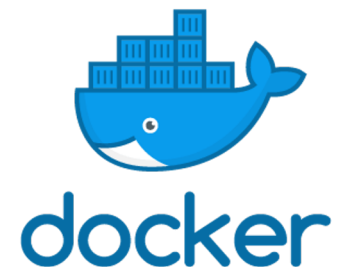
Agenda

- Qué es Docker?
- Arquitectura de Docker
- Imágenes y Contenedores
- “Dockerizando” una aplicación
- Docker Compose



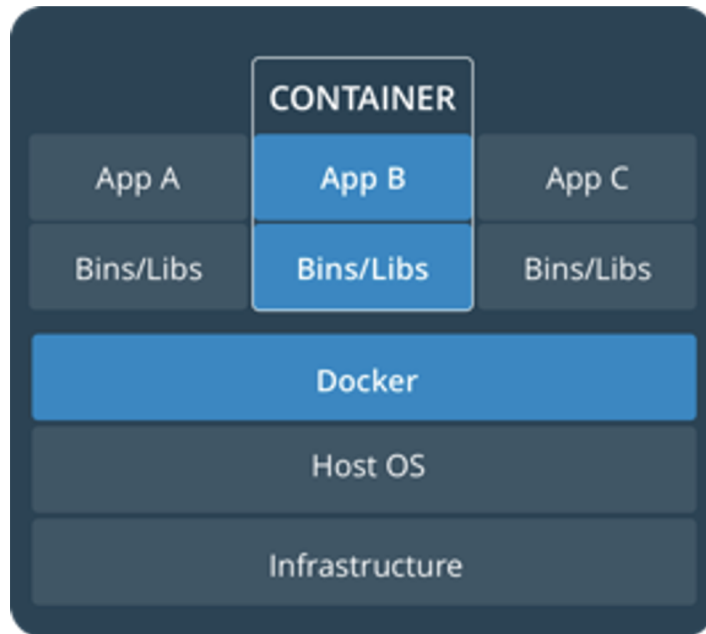
Docker

- Es una plataforma para developers y administradores que permite *desarrollar, deployar y ejecutar* aplicaciones en contenedores.
- Permite empaquetar software en conjunto con sus dependencias, crear *imágenes* y ejecutarlas en *contenedores*.
- Proporciona una capa de abstracción y automatización de aplicaciones en múltiples sistemas operativos.

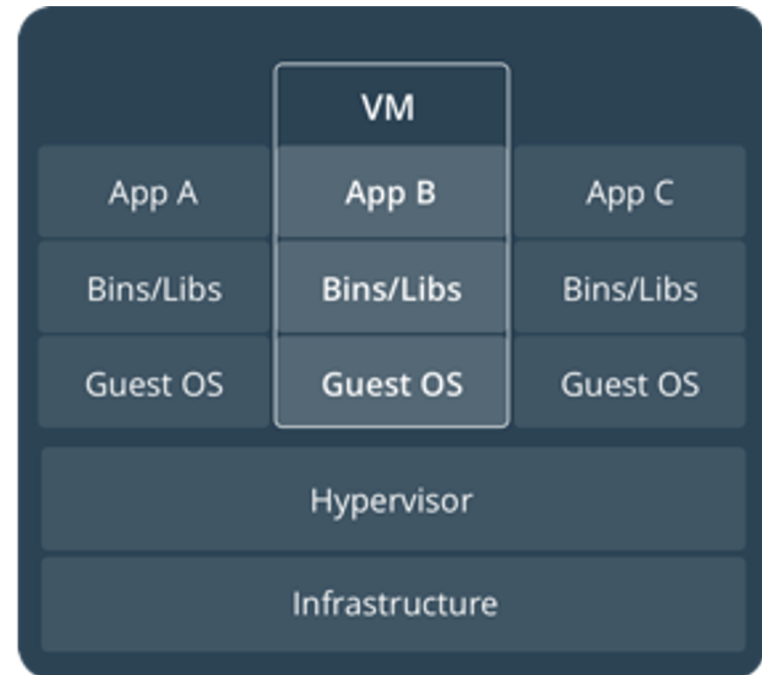




Contenedores y Máquinas Virtuales

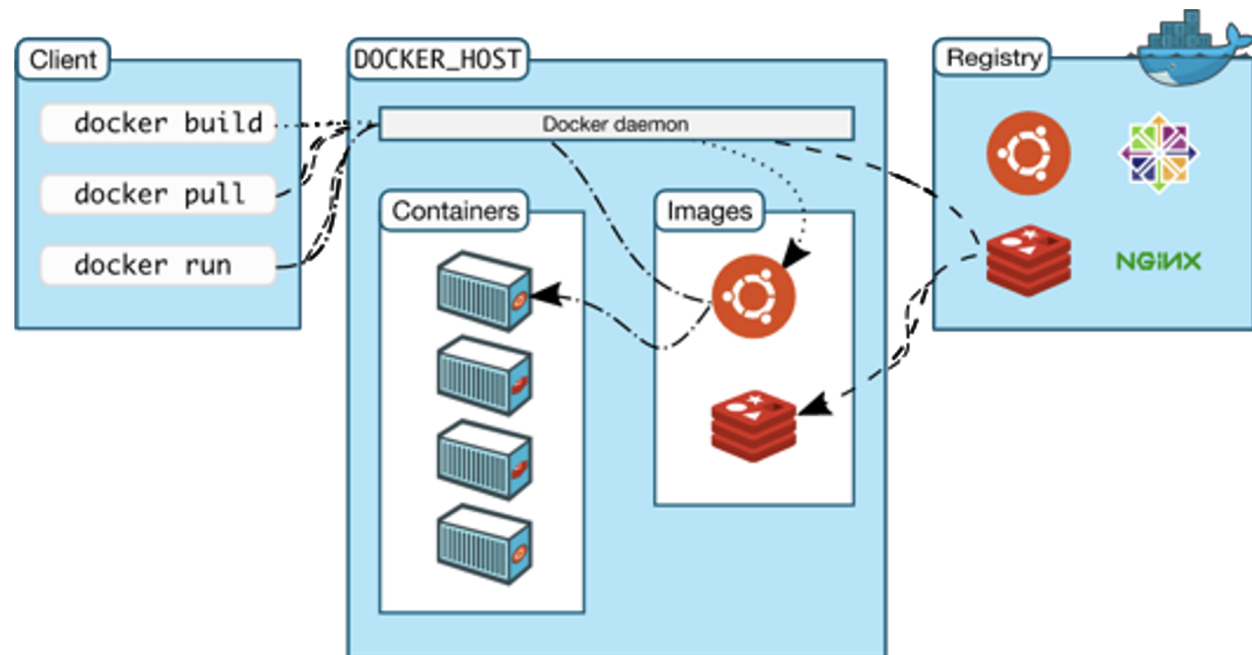


VS



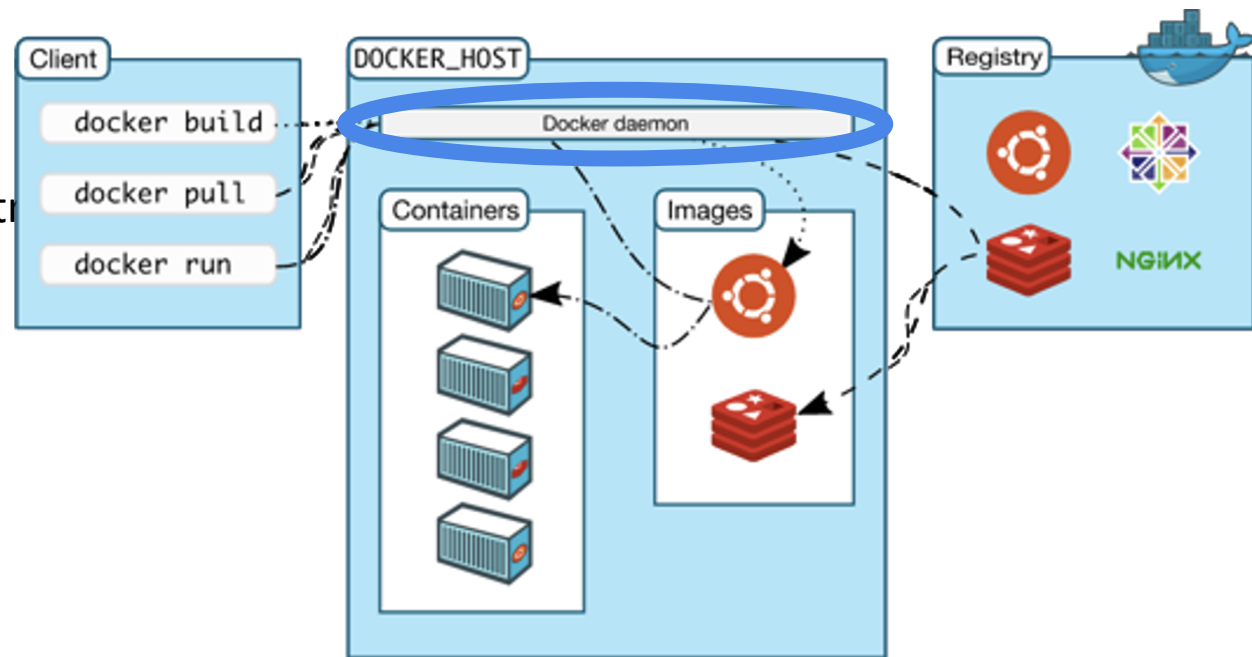
Arquitectura de Docker

Docker utiliza una arquitectura de tipo **cliente servidor** quienes se comunican utilizando una **API Rest**.

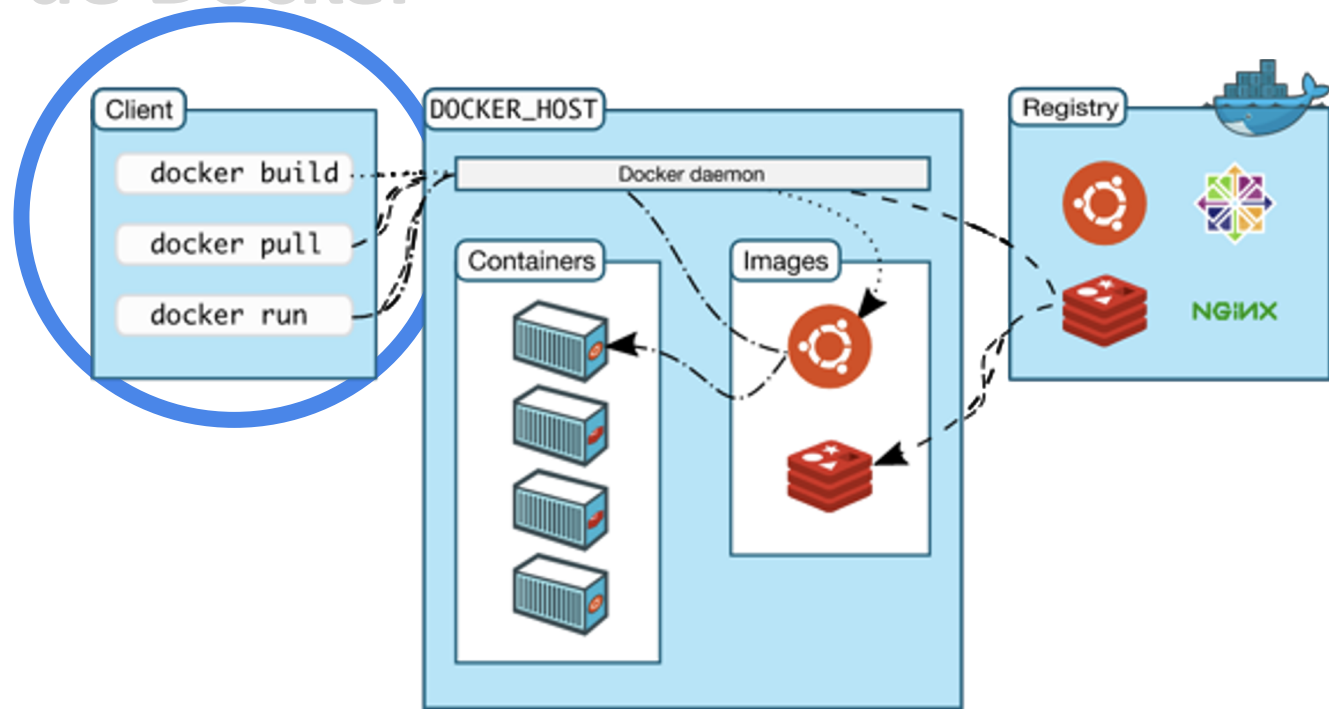


Arquitectura de Docker

Un servicio llamado **Docker Daemon** escucha peticiones y administra imágenes, contenedores, volúmenes, redes, etc.

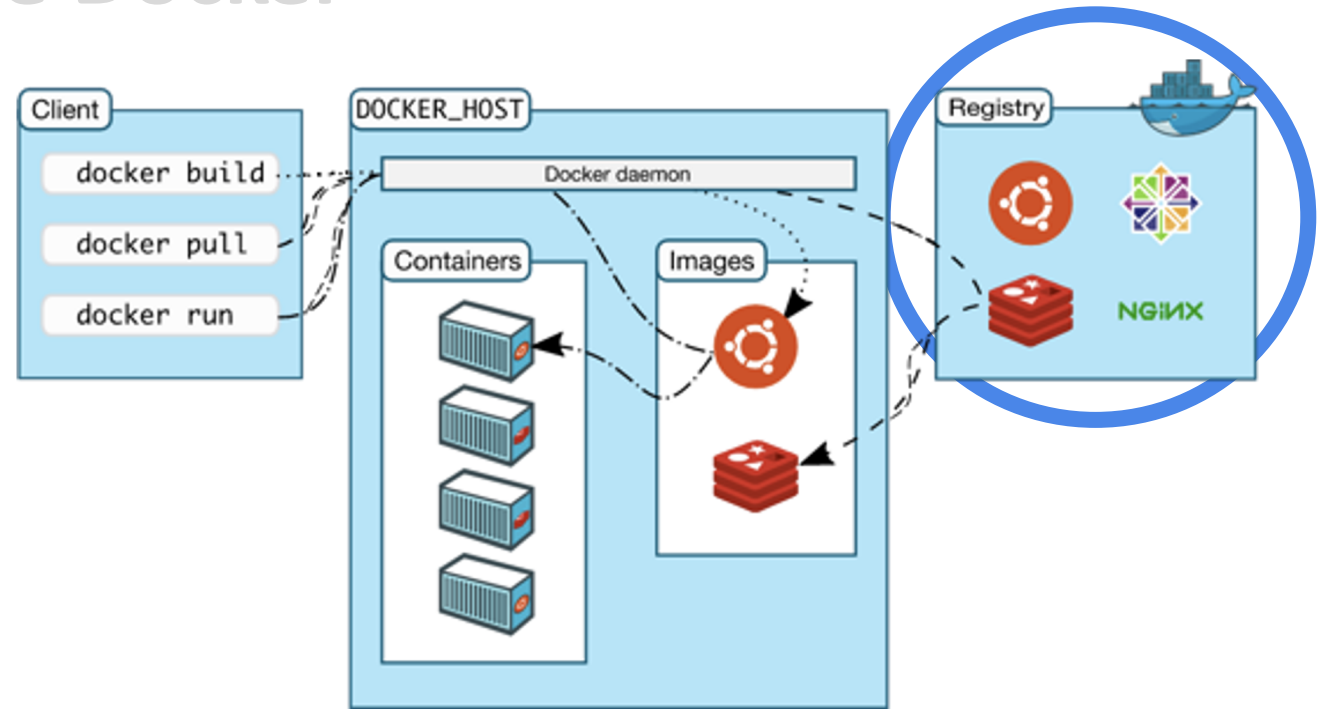


Arquitectura de Docker



Arquitectura de Docker

Las imágenes pueden almacenarse en **repositorios** públicos o privados, siendo **DockerHub** el más conocido.



Imágenes y Contenedores

- Una **imagen** es un paquete ejecutable que incluye todo lo necesario para ejecutar una aplicación:
 - código fuente o binario
 - dependencias (librerías)
 - configuraciones:
 - variables de entorno
 - archivos de configuración



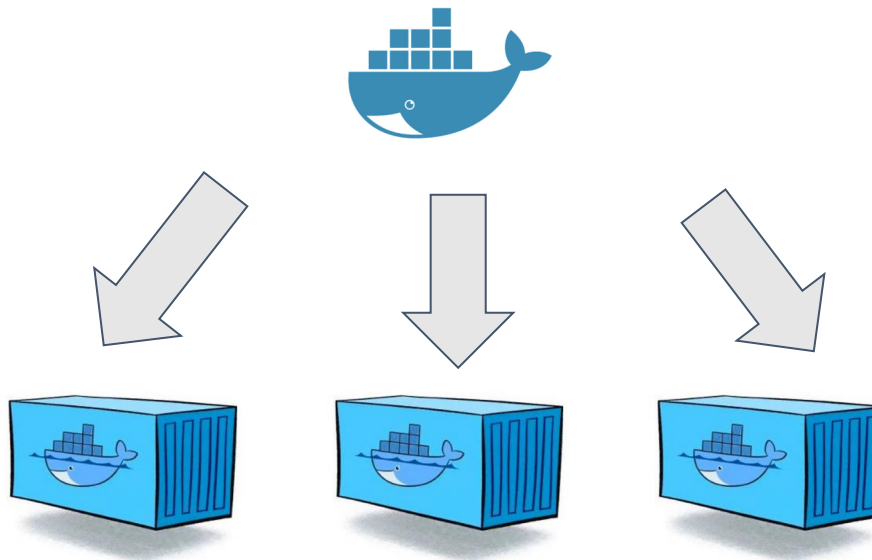
\$ docker build



Imágenes y Contenedores

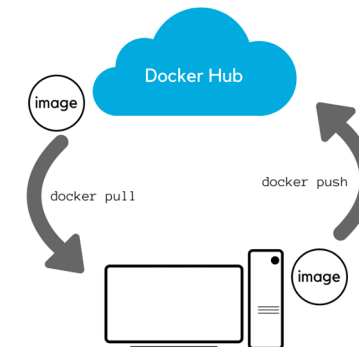
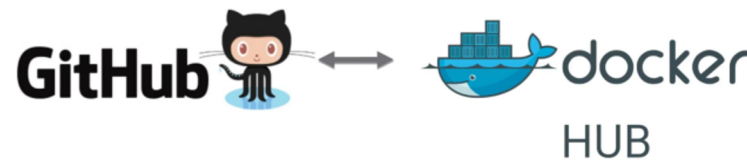
- Un **contenedor** es una instancia de una imagen en tiempo de ejecución.

\$ docker run



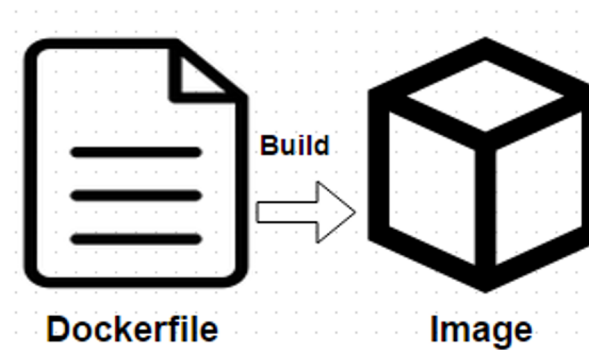
DockerHub

- Docker nos permite almacenar imágenes en repositorios centralizados.
- [DockerHub](https://hub.docker.com/) es el más popular, con más de 400K de imágenes públicas para utilizar.
- También podemos crear repositorios privados para almacenar nuestras imágenes.



“Dockerizando” una aplicación

- Existen dos formas de generar una imagen de una aplicación:
 1. Definir un archivo Dockerfile y especificar los pasos para el build.
 1. Iniciar un contenedor de una imagen existente, modificarlo y ejecutar el comando ***docker commit***.





“Dockerizando” una aplicación

En un Dockerfile:

```
FROM maven:3.6.1-jdk-12 as Builder

WORKDIR /build

COPY . /build

RUN mvn clean package

FROM openjdk:12 as Target

COPY --from=Builder /build/target/myApp.jar .

ENTRYPOINT ["java","-jar","myApp.jar"]

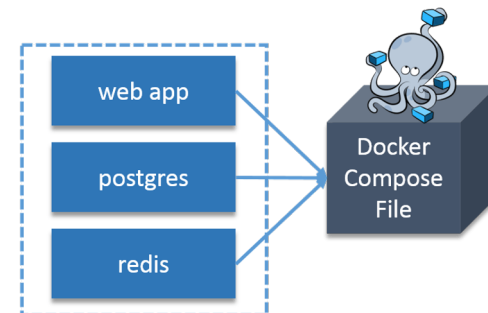
EXPOSE 8080
```

1. Definimos la **imagen** de la cual vamos a partir.
1. Especificamos **comandos** para agregar capas a dicha imagen.
1. Definimos **qué** aplicación se va a ejecutar.
1. Definimos el **puerto** que se va a exponer.



Docker Compose

- Si bien podemos utilizar un sólo contenedor para correr nuestra aplicación, no es muy escalable.
- Se suelen utilizar contenedores para distintos servicios como bases de datos, caché, servidores web, middlewares, colas de mensajería, entre otros.
- Imaginando esta situación, entendemos que manejar muchos contenedores por línea de comando resultaría tedioso.
- Para ésto tenemos ***Docker Compose***





Docker Compose

```
version: 3
services:
  myDB:
    image: mysql
    ports:
      - "3306:3306"
    environment:
      - MYSQL_USER=user
      - MYSQL_PASSWORD=password
      - MYSQL_DATABASE=mydb
  webServer:
    image: nginx
    ports:
      - "80:80"
    depends_on:
      - myDB
```

Podemos definir:

- Todos las imágenes que vamos a utilizar (propias o ya existentes).
- Dependencias entre los mismos.
- Variables de Entorno y Configuración.
- Volúmenes.
- Redes y Subredes



Comandos más utilizados

Para Imágenes	Para Contenedores
<code>docker image ls</code> (lista imágenes descargadas)	<code>docker container run <image name></code> (crea un nuevo contenedor)
<code>docker pull <image name></code> (descarga una imagen)	<code>docker container ls</code> (lista los contenedores)
<code>docker image rm <image id></code> (elimina una imagen)	<code>docker container start <container name></code> (inicia un contenedor creado)
<code>docker push <image name></code> (sube una imagen)	<code>docker container stop <container name></code> (detiene un contenedor)
<code>docker image tag <image> <name></code> (etiqueta una imagen)	<code>docker container rm <container name></code> (elimina un contenedor)

¿Dudas? ¿Consultas?

